
Universidad Aeronáutica de Querétaro

Embedded Systems Laboratory

SCADE One Laboratory Series

Practice #5

The Model as a Task: Running SCADE Under FreeRTOS

Elaborated by: Leonardo Franco García

Student ID: 12148

Date: July 23, 2026

Full name: _____

Student ID: _____

Date: _____

1 Learning Objectives

Learning Objectives

By the end of this practice, the student will be able to:

- ✓ **Run the model as a scheduled task:** execute the SCADE step function inside a dedicated periodic FreeRTOS task with `vTaskDelayUntil`, establishing the control-task architecture reused through Practice 9.
- ✓ **Fan model state out to many consumers:** distribute each cycle's output to independent consumer tasks (LEDs and UART telemetry) through separate queues, so a slow consumer never stalls the control loop.
- ✓ **Serialize a shared resource:** guard the single console with a mutex so log lines from concurrent tasks stay intact.
- ✓ **Reason about priority and cadence:** assign task priorities so the model's 100 ms period is held regardless of lower-priority I/O, and predict scheduler behaviour under load.

2 Introduction

Up to Practice 4 the SCADE model ran in a bare `while(1)` loop that did one thing per iteration: step, drive I/O, wait. A real aircraft controller cannot work that way — its control law must hold a strict cycle while telemetry, indicators, and health monitoring all run at the same time and at different rates. FreeRTOS, the kernel already beneath every ESP-IDF program, provides the missing structure: independent tasks scheduled by priority, exchanging data through queues and sharing resources through mutexes. This practice performs the pivot that the rest of the course depends on — it turns the model into *one periodic task among several*, feeding LEDs and the UART telemetry of Practice 4 as independent consumers. The same skeleton built here is what Practices 6–9 attach state machines, sensors, actuators, and a safety monitor onto, all the way to the flight control-surface controller.

3 Bill of Materials

#	Component / Software	Qty	Notes
1	SCADE One (Ansys)	1	Swan code generation enabled
2	ESP32 DevKit V1	1	Environment from Practice 2/3
3	USB-A to Micro-USB cable	1	Data cable (console)
4	ESP-IDF (v5.x)	1	FreeRTOS is built in — no extra install
5	Generated <code>scade_gen</code> component	1	Reused from Practice 3 (<code>TrafficLight</code>)
6	Three LEDs + 220 Ω resistors	3	Traffic-light LEDs (GPIO 25/26/27)
7	Jumper wire (M–M)	1	UART2 telemetry loopback: GPIO 4 to GPIO 5

No new hardware

FreeRTOS ships inside ESP-IDF, so nothing is installed and no new wiring is needed beyond what Practices 3 and 4 already used: the three LEDs on GPIO 25/26/27 and the UART2 loopback jumper on GPIO 4–GPIO 5. The on-board LED on GPIO 2 becomes a heartbeat driven by its own task.

4 Theory & Block Reference

No new SCADE modeling blocks appear here — the `TrafficLight` model is reused exactly as generated. What is new is the FreeRTOS API in the glue layer and, more importantly, the *architecture* it enables. The reference documents the three kernel primitives used, then the producer–consumer pattern that binds them into the control-task skeleton of the whole remaining course.

4.1. The scheduler, and why the model becomes a task

FreeRTOS keeps a priority-ordered list of tasks. At every tick (1 ms by default on ESP-IDF), and whenever a task blocks, it runs the highest priority task that is *ready*. A task stops being ready when it blocks on a delay, a queue, or a mutex — that is precisely what lets lower priority work proceed. Placing the SCADE step function in its own task, blocked to a fixed period, buys three things at once: the control cadence becomes independent of how long I/O takes, the model can feed several consumers without knowing they exist, and the timing is now owned by the kernel rather than by a hand-tuned `vTaskDelay` at the bottom of a monolithic loop.

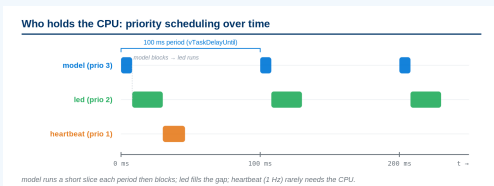
Primitive: `xTaskCreate` / `vTaskDelayUntil`

Figure 1: Tasks at three priorities scheduled on one core; the control task wakes on a fixed period — annotated view.

Key elements:

- **`xTaskCreate(fn, name, stack, arg, prio, handle)`**: spawns a task running `fn` forever, with its own stack.
- **Stack depth**: the model task needs more because the SCADE context lives on its stack.
- **Priority**: higher number = more urgent; equal priorities time-slice.
- **`vTaskDelayUntil(&last, period)`**: wakes at a fixed *period*, absorbing the task's own run time — unlike `vTaskDelay`, which sleeps *for* a duration and therefore drifts.

Behavior:

The control task must use `vTaskDelayUntil` so its 100 ms tick does not accumulate the time spent framing, sending, or logging. This is what makes the cadence deterministic.

Primitive: xQueueCreate / Send / Receive

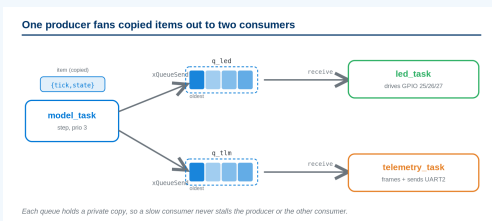


Figure 2: One producer sending fixed-size items to two consumers through two queues — annotated view.

Key elements:

- **xQueueCreate(len, itemSize):** a thread-safe FIFO of fixed-size items.
- **xQueueSend(q, &item, wait):** copies an item in; **wait=0** means never block the producer.
- **xQueueReceive(q, &item, wait):** copies the next item out; **portMAX_DELAY** blocks until one arrives.

Behavior:

A queue *copies* items, so producer and consumer never share memory — no race. A single producer can drive many consumers by giving each its own queue and sending to all of them each cycle. A full queue means that consumer is too slow; with **wait=0** the control task drops the item rather than stall.

Primitive: xSemaphoreCreateMutex / Take / Give

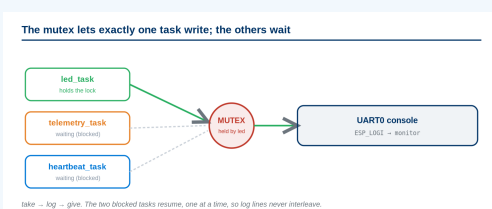


Figure 3: Three tasks serialising the shared console through one mutex — annotated view.

Key elements:

- **xSemaphoreCreateMutex():** a lock exactly one task can hold at a time.
- **xSemaphoreTake(m, wait)** / **xSemaphoreGive(m):** acquire and release.

Behavior:

The console (UART0) is one shared resource; concurrent **ESP_LOGI** calls interleave their characters. Wrapping each log in take/give serialises access. Give the mutex back on every path and never block while holding it, or the other tasks wait forever.

4.2. The control-task pattern (used through Practice 9)

The architecture built in the Main Activity is deliberately the template for the rest of the course, drawn in Figure 4: a single *control task* runs the model on a strict period and pushes each result into one queue per consumer; independent consumer tasks block on their queue and act — driving LEDs, framing telemetry — at their own pace; a low-priority heartbeat proves the system is alive; and a mutex keeps the shared console coherent. In Practices 7–9 the same shape gains a sensor task feeding the model and an actuator task driven by it, but the skeleton does not change.

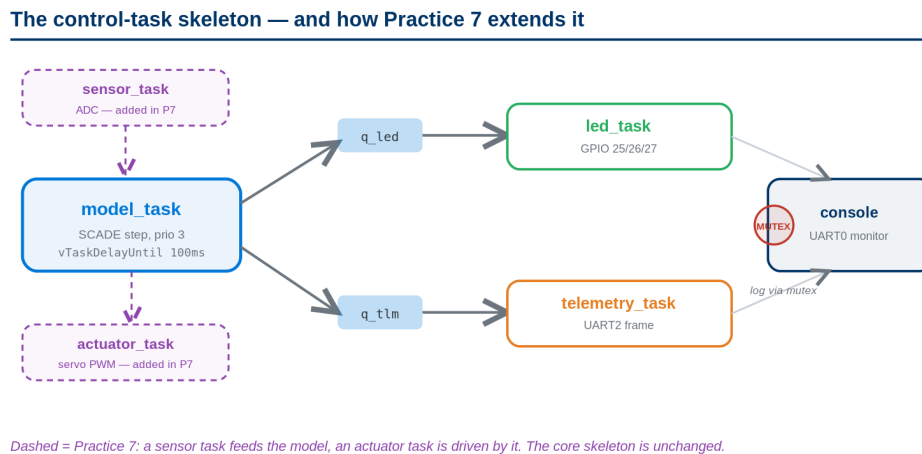


Figure 4: The control-task skeleton: one periodic model task fans state out to LED and telemetry consumers; a heartbeat runs lowest; a mutex guards the console.

5 Worked Example

5.1. Objective

Build the smallest version of the pattern: one producer task steps the model and one consumer task drives the LEDs, communicating only through a queue. This isolates `xTaskCreate`, `vTaskDelayUntil`, and a single queue before the Main Activity fans out to a second consumer and adds the mutex. About 25 minutes.

5.2. Step-by-Step

1. Create the project from Practice 4.

Reuse `lg_telemetry` (or `lg_bridge`) so the generated component travels with it:

```
cd C:\esp
xcopy /e /i lg_telemetry lg_rtos
cd lg_rtos
idf.py fullclean
```

2. Write a producer and a consumer.

Open notepad `main\main.c` and paste:

```
#include "freertos/FreeRTOS.h"
```

```

#include "freertos/task.h"
#include "freertos/queue.h"
#include "driver/gpio.h"
#include "esp_log.h"
#include "operator1_module0.h" // generated (TrafficLight)

#define LED_RED 25
#define LED_YEL 26
#define LED_GRN 27
static const char *TAG = "RTOS";

typedef struct { int tick; uint8_t state; } lg_msg_t; // b0=G b1=Y b2=R
static QueueHandle_t q;

static void model_task(void *arg) // producer, priority 3
{
    outC_operator1_module0 ctx;
    swan_int32 cont; swan_bool g, y, r;
    operator1_reset_module0(&ctx);
    TickType_t last = xTaskGetTickCount();
    while (1) {
        operator1_module0(&cont, &g, &y, &r, &ctx);
        lg_msg_t m = { .tick = (int)cont,
                       .state = (g?1:0) | (y?2:0) | (r?4:0) };
        xQueueSend(q, &m, 0);
        vTaskDelayUntil(&last, pdMS_TO_TICKS(100)); // strict period
    }
}

static void led_task(void *arg) // consumer, priority 2
{
    gpio_set_direction(LED_RED, GPIO_MODE_OUTPUT);
    gpio_set_direction(LED_YEL, GPIO_MODE_OUTPUT);
    gpio_set_direction(LED_GRN, GPIO_MODE_OUTPUT);
    lg_msg_t m;
    while (1) {
        if (xQueueReceive(q, &m, portMAX_DELAY) == pdTRUE) {
            gpio_set_level(LED_GRN, m.state & 1);
            gpio_set_level(LED_YEL, (m.state >> 1) & 1);
            gpio_set_level(LED_RED, (m.state >> 2) & 1);
            ESP_LOGI(TAG, "LED tick=%d G=%d Y=%d R=%d", m.tick,
                     m.state & 1, (m.state >> 1) & 1, (m.state >> 2) & 1);
        }
    }
}

void app_main(void)
{
    q = xQueueCreate(8, sizeof(lg_msg_t));
    xTaskCreate(model_task, "model", 3072, NULL, 3, NULL);
    xTaskCreate(led_task, "led", 3072, NULL, 2, NULL);
}

```

3. Set the dependencies.

Edit `main\CMakeLists.txt` (FreeRTOS needs no entry — it is core):

```

idf_component_register(SRCS "main.c"
                      INCLUDE_DIRS "."
                      REQUIRES scade_gen esp_driver_gpio)

```

4. Build, flash, and verify.

Run `idf.py -p COM8 flash monitor`. The LEDs cycle $G \rightarrow Y \rightarrow R$ exactly as in Practice 3, but now the model and the LED driver are two separate tasks joined only by a queue — the producer never calls the consumer.

```

I (264) main_task: Started on CPU0
I (274) main_task: Calling app_main()
I (274) RTOS: LED tick=0 G=1 Y=0 R=0
I (274) main_task: Returned from app_main()
I (374) RTOS: LED tick=1 G=1 Y=0 R=0
I (474) RTOS: LED tick=2 G=1 Y=0 R=0
I (574) RTOS: LED tick=3 G=1 Y=0 R=0
I (674) RTOS: LED tick=4 G=1 Y=0 R=0
I (774) RTOS: LED tick=5 G=1 Y=0 R=0
I (874) RTOS: LED tick=6 G=1 Y=0 R=0
I (974) RTOS: LED tick=7 G=1 Y=0 R=0
I (1074) RTOS: LED tick=8 G=1 Y=0 R=0
I (1174) RTOS: LED tick=9 G=1 Y=0 R=0
I (1274) RTOS: LED tick=10 G=1 Y=0 R=0
I (1374) RTOS: LED tick=11 G=1 Y=0 R=0
I (1474) RTOS: LED tick=12 G=1 Y=0 R=0
I (1574) RTOS: LED tick=13 G=1 Y=0 R=0
I (1674) RTOS: LED tick=14 G=1 Y=0 R=0
I (1774) RTOS: LED tick=15 G=1 Y=0 R=0
I (1874) RTOS: LED tick=16 G=1 Y=0 R=0
I (1974) RTOS: LED tick=17 G=1 Y=0 R=0
I (2074) RTOS: LED tick=18 G=1 Y=0 R=0
I (2174) RTOS: LED tick=19 G=1 Y=0 R=0
I (2274) RTOS: LED tick=20 G=1 Y=0 R=0
I (2374) RTOS: LED tick=21 G=1 Y=0 R=0
I (2474) RTOS: LED tick=22 G=1 Y=0 R=0
I (2574) RTOS: LED tick=23 G=1 Y=0 R=0
I (2674) RTOS: LED tick=24 G=1 Y=0 R=0
I (2774) RTOS: LED tick=25 G=1 Y=0 R=0
I (2874) RTOS: LED tick=26 G=1 Y=0 R=0
I (2974) RTOS: LED tick=27 G=1 Y=0 R=0
I (3074) RTOS: LED tick=28 G=1 Y=0 R=0
I (3174) RTOS: LED tick=29 G=1 Y=0 R=0
I (3274) RTOS: LED tick=30 G=1 Y=0 R=0
I (3374) RTOS: LED tick=31 G=1 Y=0 R=0

```

Figure 5: Producer/consumer split: the model task feeds the LED task through a queue.

6 Main Activity

6.1. Description

Grow the worked example into the full control-task skeleton. The model task now fans its state out to *two* independent consumers through two queues: the LED task of the example, and a telemetry task that rebuilds the four-byte UART frame of Practice 4 and validates it on the GPIO 4–5 loopback. A heartbeat task runs at the lowest priority, and a single mutex guards every console write so the three talking tasks never garble each other. The model still steps once per 100 ms cycle — now feeding a small concurrent system.

6.2. Functional Specifications

- **Spec 1:** `app_main` creates four tasks — `model` (prio 3), `led` (prio 2), `telemetry` (prio 2), `heartbeat` (prio 1) — and two queues plus one mutex.

- **Spec 2:** The model task steps `TrafficLight` on a fixed 100 ms period with `vTaskDelayUntil` and sends the same `{tick,state}` item to both queues with a non-blocking send.
- **Spec 3:** The LED task drives GPIO 25/26/27 from its queue; the telemetry task frames `[0xAA][tick][state][chk]`, sends it on UART2, reads it back, and validates header and checksum (Practice 4).
- **Spec 4:** Every `ESP_LOGI` is wrapped by the shared mutex; no two log lines interleave.
- **Spec 5:** The heartbeat task blinks GPIO 2 and logs once per second at the lowest priority without disturbing the 100 ms model cadence.

6.3. Activity Steps

1. Confirm the generated interface.

As before, confirm in `components\scade_gen\operator1_module0.h` the step signature `operator1_module0(&cont, &green, &yellow, &red, &ctx)` and context type `outC_operator1_module0`.

2. Write the full multi-task glue.

Replace `main.c` with the program below.

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "freertos/semphr.h"
#include "driver/gpio.h"
#include "driver/uart.h"
#include "esp_log.h"
#include "operator1_module0.h" // generated (TrafficLight)

#define LED_RED 25
#define LED_YEL 26
#define LED_GRN 27
#define LED_HB 2
#define U2_PORT UART_NUM_2
#define U2_TX 4
#define U2_RX 5
#define HDR 0xAA
static const char *TAG = "RTOS";

typedef struct { int tick; uint8_t state; } lg_msg_t; // b0=G b1=Y b2=R
static QueueHandle_t q_led, q_tlm;
static SemaphoreHandle_t logmux;

// ---- control task: the model, on a strict 100 ms period ----
static void model_task(void *arg)
{
    outC_operator1_module0 ctx; swan_int32 cont; swan_bool g,y,r;
    operator1_reset_module0(&ctx);
    TickType_t last = xTaskGetTickCount();
    while (1) {
        operator1_module0(&cont, &g, &y, &r, &ctx);
        lg_msg_t m = { .tick=(int)cont, .state=(g?1:0)|(y?2:0)|(r?4:0) };
        xQueueSend(q_led, &m, 0); // fan out to both consumers
        xQueueSend(q_tlm, &m, 0);
        vTaskDelayUntil(&last, pdMS_TO_TICKS(100));
    }
}

// ---- consumer 1: LEDs ----
static void led_task(void *arg)
{
    gpio_set_direction(LED_RED, GPIO_MODE_OUTPUT);
    gpio_set_direction(LED_YEL, GPIO_MODE_OUTPUT);
```

```

    gpio_set_direction(LED_GRN, GPIO_MODE_OUTPUT);
    lg_msg_t m;
    while (1) if (xQueueReceive(q_led, &m, portMAX_DELAY) == pdTRUE) {
        gpio_set_level(LED_GRN, m.state & 1);
        gpio_set_level(LED_YEL, (m.state >> 1) & 1);
        gpio_set_level(LED_RED, (m.state >> 2) & 1);
        xSemaphoreTake(logmux, portMAX_DELAY);
        ESP_LOGI(TAG, "LED tick=%d G=%d Y=%d R=%d", m.tick,
                  m.state&1, (m.state>>1)&1, (m.state>>2)&1);
        xSemaphoreGive(logmux);
    }
}

// ---- consumer 2: UART telemetry with loopback validation (Practice 4) ----
static void telemetry_task(void *arg)
{
    uart_config_t cfg = { .baud_rate=115200, .data_bits=UART_DATA_8_BITS,
                          .parity=UART_PARITY_DISABLE, .stop_bits=UART_STOP_BITS_1,
                          .flow_ctrl=UART_HW_FLOWCTRL_DISABLE, .source_clk=UART_SCLK_DEFAULT };
    uart_driver_install(U2_PORT, 256, 0, 0, NULL, 0);
    uart_param_config(U2_PORT, &cfg);
    uart_set_pin(U2_PORT, U2_TX, U2_RX, UART_PIN_NO_CHANGE, UART_PIN_NO_CHANGE);
    lg_msg_t m;
    while (1) if (xQueueReceive(q_tlm, &m, portMAX_DELAY) == pdTRUE) {
        uint8_t tx[4] = { HDR, (uint8_t)m.tick, m.state, 0 };
        tx[3] = tx[0]^tx[1]^tx[2];
        uart_write_bytes(U2_PORT, tx, 4);
        uint8_t rx[4] = {0};
        int n = uart_read_bytes(U2_PORT, rx, 4, pdMS_TO_TICKS(20));
        uint8_t chk = rx[0]^rx[1]^rx[2];
        xSemaphoreTake(logmux, portMAX_DELAY);
        if (n==4 && rx[0]==HDR && rx[3]==chk)
            ESP_LOGI(TAG, "TLM tick=%d ok", rx[1]);
        else
            ESP_LOGW(TAG, "TLM bad n=%d", n);
        xSemaphoreGive(logmux);
    }
}

// ---- lowest priority: heartbeat ----
static void heartbeat_task(void *arg)
{
    gpio_set_direction(LED_HB, GPIO_MODE_OUTPUT);
    bool s = false;
    while (1) {
        s = !s; gpio_set_level(LED_HB, s);
        xSemaphoreTake(logmux, portMAX_DELAY);
        ESP_LOGI(TAG, "HB alive");
        xSemaphoreGive(logmux);
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

void app_main(void)
{
    q_led = xQueueCreate(8, sizeof(lg_msg_t));
    q_tlm = xQueueCreate(8, sizeof(lg_msg_t));
    logmux = xSemaphoreCreateMutex();
    xTaskCreate(model_task, "model", 3072, NULL, 3, NULL);
    xTaskCreate(led_task, "led", 3072, NULL, 2, NULL);
    xTaskCreate(telemetry_task, "tlm", 3072, NULL, 2, NULL);
    xTaskCreate(heartbeat_task, "hb", 2560, NULL, 1, NULL);
}

```

3. Set the dependencies.

The glue now uses GPIO and UART:

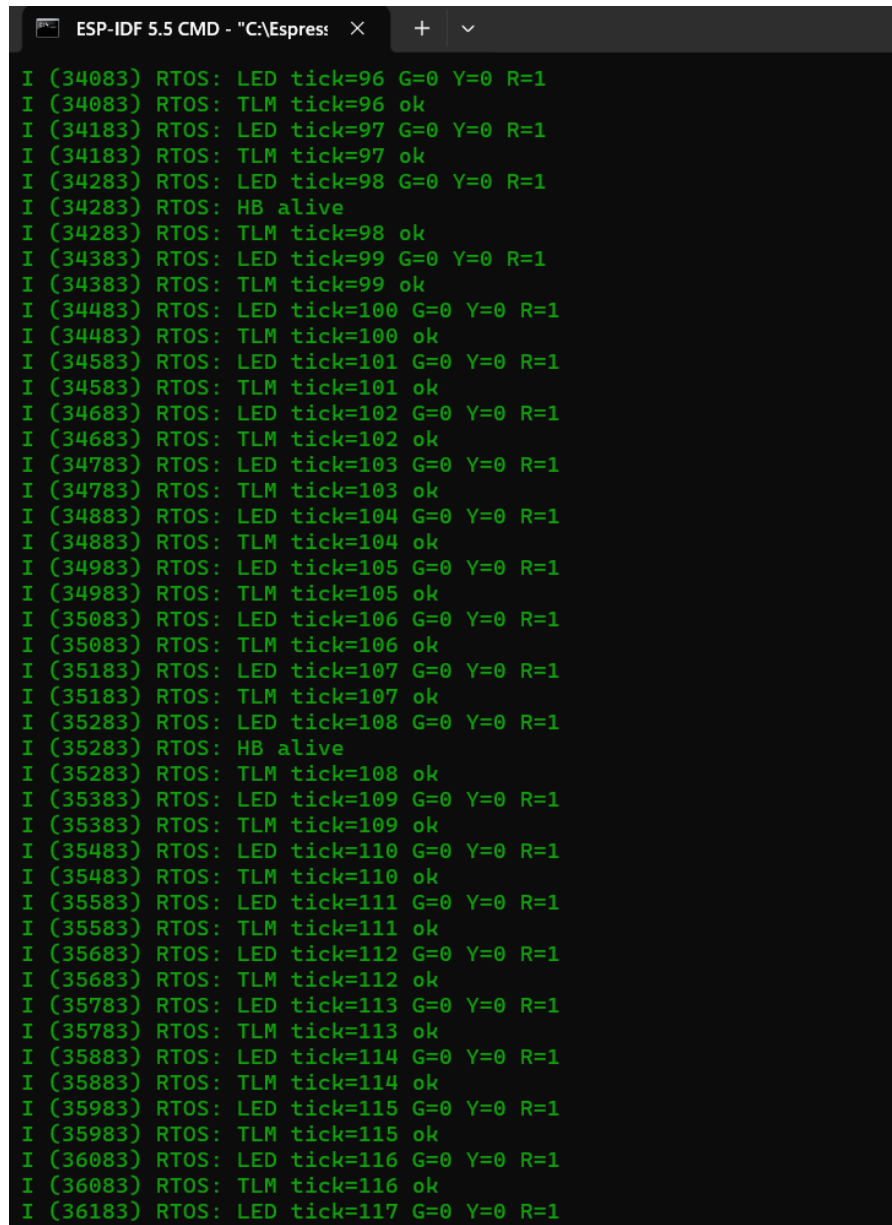
```

idf_component_register(SRCS "main.c"
                      INCLUDE_DIRS "."
                      REQUIRES scade_gen esp_driver_gpio esp_driver_uart)

```

4. Build, flash, and verify the concurrency.

With the GPIO 4–5 jumper in place, run `idf.py -p COM8 flash monitor`. Confirm: the LEDs cycle on the 5/2/5s schedule; LED and TLM lines appear together each cycle with matching ticks; an HB **alive** line appears once per second; and no line is ever garbled. Pulling the jumper turns the TLM lines to **bad** while the LEDs and heartbeat keep running — proof the tasks are independent.



```

ESP-IDF 5.5 CMD - "C:\Espress
+  v
I (34083) RTOS: LED tick=96 G=0 Y=0 R=1
I (34083) RTOS: TLM tick=96 ok
I (34183) RTOS: LED tick=97 G=0 Y=0 R=1
I (34183) RTOS: TLM tick=97 ok
I (34283) RTOS: LED tick=98 G=0 Y=0 R=1
I (34283) RTOS: HB alive
I (34283) RTOS: TLM tick=98 ok
I (34383) RTOS: LED tick=99 G=0 Y=0 R=1
I (34383) RTOS: TLM tick=99 ok
I (34483) RTOS: LED tick=100 G=0 Y=0 R=1
I (34483) RTOS: TLM tick=100 ok
I (34583) RTOS: LED tick=101 G=0 Y=0 R=1
I (34583) RTOS: TLM tick=101 ok
I (34683) RTOS: LED tick=102 G=0 Y=0 R=1
I (34683) RTOS: TLM tick=102 ok
I (34783) RTOS: LED tick=103 G=0 Y=0 R=1
I (34783) RTOS: TLM tick=103 ok
I (34883) RTOS: LED tick=104 G=0 Y=0 R=1
I (34883) RTOS: TLM tick=104 ok
I (34983) RTOS: LED tick=105 G=0 Y=0 R=1
I (34983) RTOS: TLM tick=105 ok
I (35083) RTOS: LED tick=106 G=0 Y=0 R=1
I (35083) RTOS: TLM tick=106 ok
I (35183) RTOS: LED tick=107 G=0 Y=0 R=1
I (35183) RTOS: TLM tick=107 ok
I (35283) RTOS: LED tick=108 G=0 Y=0 R=1
I (35283) RTOS: HB alive
I (35283) RTOS: TLM tick=108 ok
I (35383) RTOS: LED tick=109 G=0 Y=0 R=1
I (35383) RTOS: TLM tick=109 ok
I (35483) RTOS: LED tick=110 G=0 Y=0 R=1
I (35483) RTOS: TLM tick=110 ok
I (35583) RTOS: LED tick=111 G=0 Y=0 R=1
I (35583) RTOS: TLM tick=111 ok
I (35683) RTOS: LED tick=112 G=0 Y=0 R=1
I (35683) RTOS: TLM tick=112 ok
I (35783) RTOS: LED tick=113 G=0 Y=0 R=1
I (35783) RTOS: TLM tick=113 ok
I (35883) RTOS: LED tick=114 G=0 Y=0 R=1
I (35883) RTOS: TLM tick=114 ok
I (35983) RTOS: LED tick=115 G=0 Y=0 R=1
I (35983) RTOS: TLM tick=115 ok
I (36083) RTOS: LED tick=116 G=0 Y=0 R=1
I (36083) RTOS: TLM tick=116 ok
I (36183) RTOS: LED tick=117 G=0 Y=0 R=1

```

Figure 6: Four tasks running concurrently: model-driven LEDs, validated telemetry, and a once-a-second heartbeat, every line intact.

7 Expected Results

Expected Results

- **SCADE Simulation:** Unchanged from Practice 3 — `cont` ramps 0–119 and wraps, exactly one of `green/yellow/red` true per cycle. Only the model's execution context changes, not its logic.
- **Console / UART Output:** Interleaved but whole lines from four tasks; LED and TLM share each tick, HB appears once per second:

```
I (1002) RTOS: LED tick=10 G=1 Y=0 R=0
I (1003) RTOS: TLM tick=10 ok
I (1200) RTOS: HB alive
I (5302) RTOS: LED tick=50 G=0 Y=1 R=0
I (5303) RTOS: TLM tick=50 ok
```
- **Physical Behavior:** The three traffic-light LEDs cycle green (5s) → yellow (2s) → red (5s), the on-board LED blinks at 1 Hz, and telemetry validates every cycle — three concurrent behaviours at three priorities from one unchanged model.

8 Assessment Questionnaire

Answer each question based on your specific implementation. Justify your responses with references to your design decisions.

Q1. [*Comprehension*] The model task uses `vTaskDelayUntil` and sends to both queues with `wait=0`. Explain how `vTaskDelayUntil` keeps the 100ms period even when the two sends and the step take variable time, and why a non-blocking send is the correct choice for a control task.

Q2. [*Analysis*] Suppose the telemetry task blocks for 2s (a stuck UART). Trace what happens to `q_tlm`, to the model task's sends, and to the LED task. Explain why the LED behaviour is unaffected — i.e. why fan-out through separate queues isolates a slow consumer.

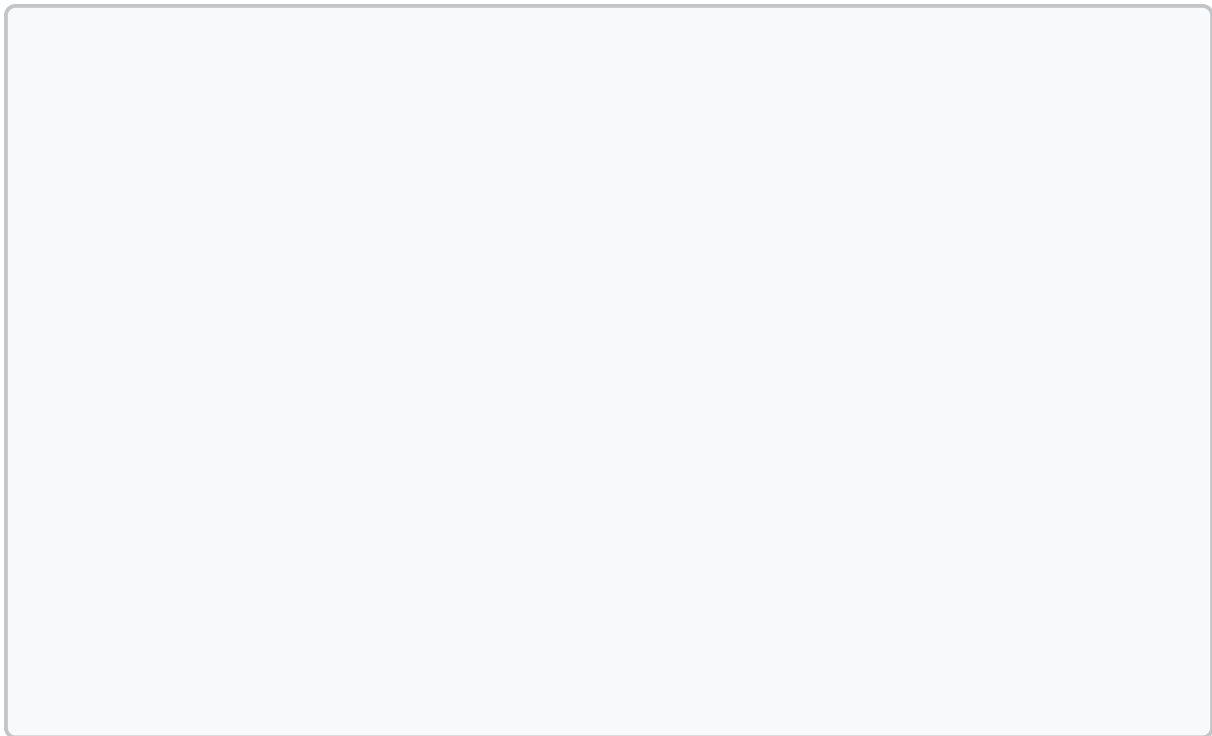
Q3. [*Analysis*] Both the LED and telemetry tasks run at priority 2, while the model is at 3 and the heartbeat at 1. Justify this ordering. What could go wrong if the telemetry task were given priority 3, equal to the model?

Q4. [*Synthesis*] Practice 7 will add a `sensor_task` that reads a potentiometer and must feed the model a `mode` input each cycle. Describe where it fits in this skeleton: its priority relative to the model, how it should hand the value over (queue, notification, or shared value with mutex), and why the model task should not read the ADC directly.

Q5. [*Critical*] Removing `vTaskDelayUntil` from the model task makes it always ready at priority 3. Describe the two failures that follow — starvation of the priority-2 and 1 tasks, and the ESP-IDF task watchdog — and explain why, in a safety-relevant controller, an unbounded highest-priority loop is itself the hazard.

9 Conclusion

Write a concise conclusion reflecting on what you implemented, what you observed, and what challenges you encountered. Connect your findings to model-based design for critical systems — in particular, what it means that the same unchanged model moved from a bare loop into a scheduled control task feeding several consumers, and why this skeleton is the right foundation for adding sensors, actuators, and a safety monitor in the practices ahead.



10 Bonus Challenge

★ Bonus Challenge

Challenge:

Instrument the scheduler. Add a fifth, lowest-priority `stats_task` that every 5s prints, under the mutex, the high-water mark of each task's stack (`uxTaskGetStackHighWaterMark`) and the number of items waiting in each queue (`uxQueueMessagesWaiting`). Use the numbers to right-size every stack and to prove the queues never back up — the same evidence you would show in a design review.

Hint:

Give each task a handle in `app_main` (the last `xTaskCreate` argument) so `stats_task` can query them. A high-water mark close to zero means that stack is nearly overflowing; raise its depth until there is comfortable margin.

11 Troubleshooting

This section is populated with real issues encountered during development. If you run into a problem not listed here, document it using the same format below.

Issue: Boot loops with “Task watchdog got triggered”

Symptom: The monitor prints `task_wdt: Task watchdog got triggered` naming the IDLE task, then the chip resets.

Cause: A high-priority task never blocks — usually the periodic delay was removed from the model loop — so lower-priority and idle tasks never run.

Fix: Every task must yield: keep `vTaskDelayUntil` in the model task and the blocking `xQueueReceive` in the consumers. An always-ready task is a design error.

Issue: Random resets: “Stack canary watchpoint triggered”

Symptom: A task crashes with a stack-overflow message naming one of your tasks.

Cause: The stack (third `xTaskCreate` argument) is too small — most often the model task, whose SCADE context lives on its stack, or a task calling `ESP_LOGI`.

Fix: Raise the depth (e.g. 2048 → 3072). Use the Bonus `stats_task` to size every stack from its high-water mark.

Issue: Log lines come out garbled or interleaved

Symptom: LED, TLM, and HB lines are spliced into one another.

Cause: A task called `ESP_LOGI` without holding the mutex, or the mutex was created after a task already started.

Fix: Wrap every log in take/give, and create `logmux` (and both queues) in `app_main` *before* any `xTaskCreate`.

Issue: Telemetry always bad, but LEDs are fine

Symptom: LED lines are correct while every TLM line reports bad.

Cause: This is the Practice 4 loopback again: the GPIO 4–5 jumper is missing or on the wrong pins, or `esp_driver_uart` is not in `REQUIRES`. The queues and tasks are working — only the UART path is broken.

Fix: Refit a solid GPIO 4–GPIO 5 jumper and confirm the `REQUIRES` list. The independence of LEDs and telemetry is exactly the fan-out property being demonstrated.